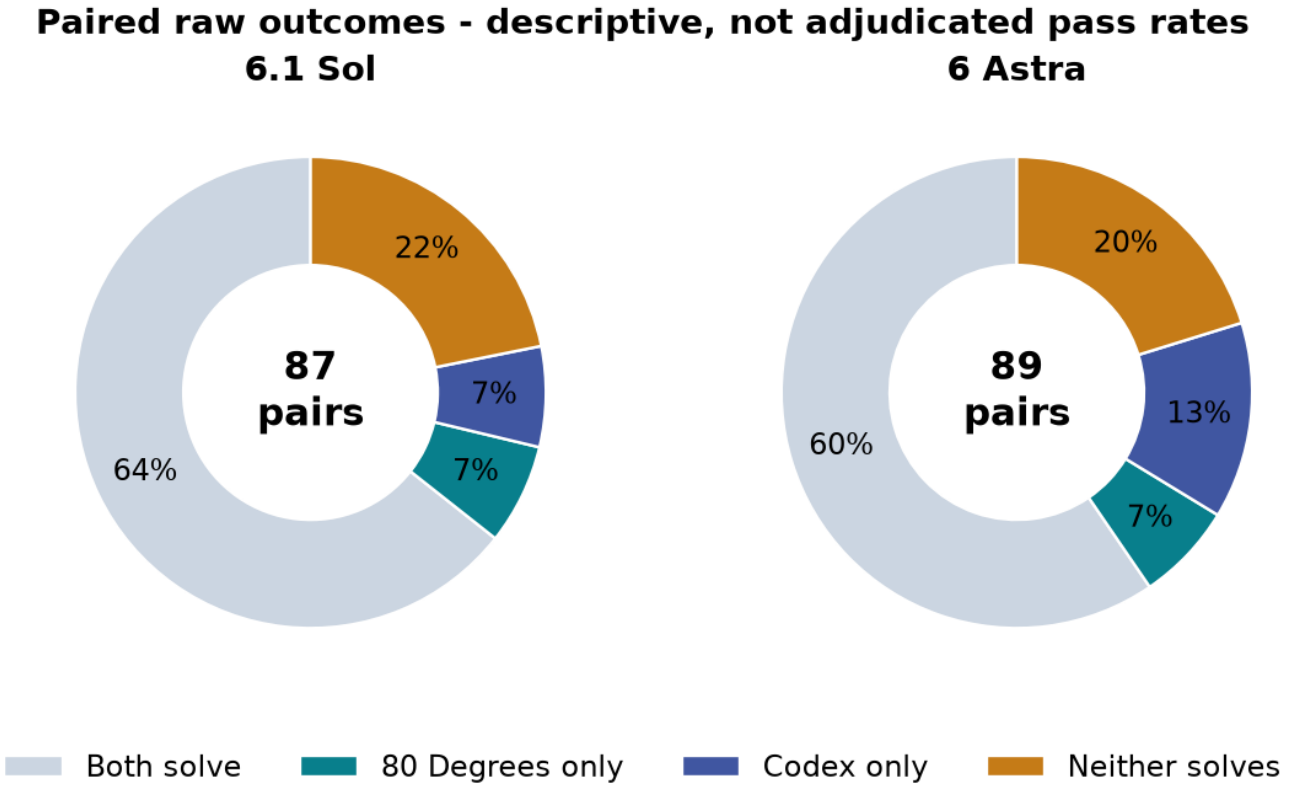
80 Degrees
Harness refinement

Engineering evidence and next experiments | 28 October 2026

Observed result: Sol draws; Astra favors Codex on raw correctness. 80 Degrees uses fewer raw complete-pair tokens, with Standard API-equivalent cost estimates 1.6% / 12.1% lower. The comparable-correctness gate is not met. On both-solved pairs, modeled costs favor Codex. Fix completion reliability before claiming equal-quality efficiency.



Raw paired gaps: Sol has six wins and six losses; Astra six wins and 12 losses. Some discordances are verifier failures, so they are not all harness mistakes. Native wins that have valid paired verification must remain regression cases while fixing weaknesses.

Priority order: (1) reconcile verifier availability and improve evidence collection; (2) prevent completion while required work is unfinished; (3) handle task ambiguity and output contracts; (4) investigate token regressions on solved tasks; (5) validate changes on an independent holdout.

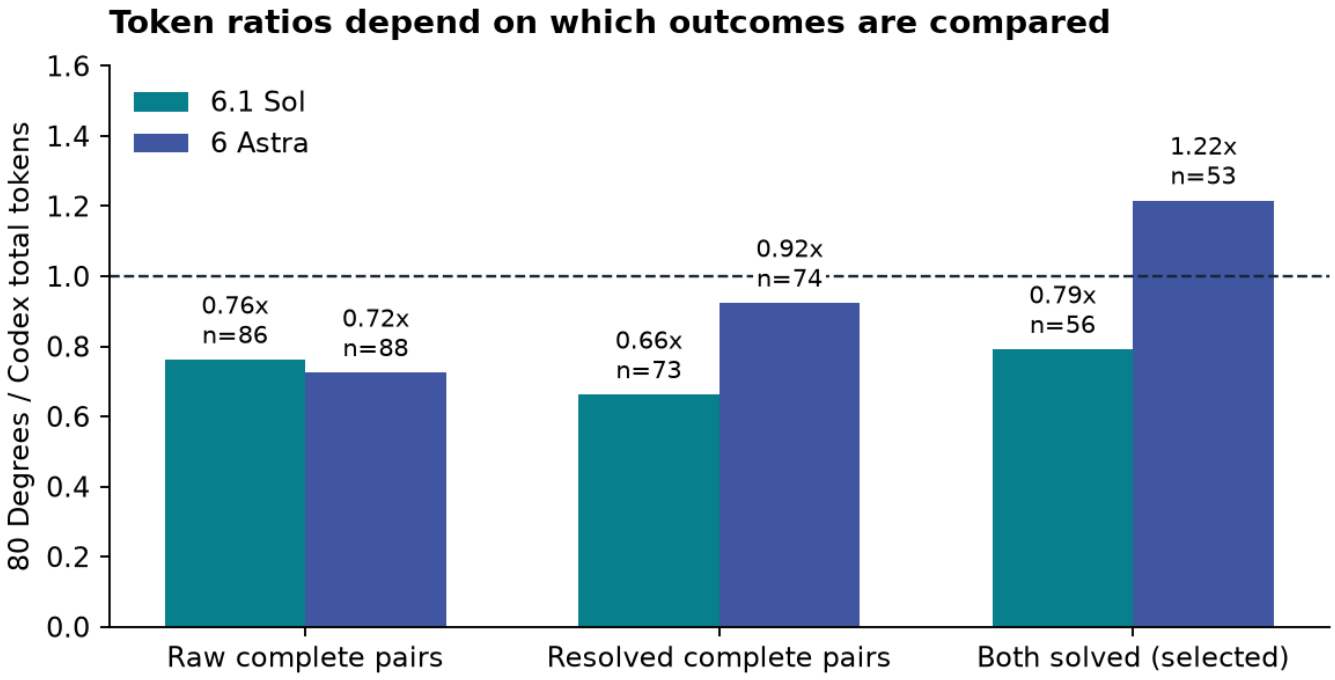
These tasks become development data once their traces guide tuning. Do not claim a rerun on them is untouched validation. No harness code or frozen agent prompts were changed for this analysis.

Observed failures, bounded hypotheses

Evidence	Hypothesis and experiment
Sol: compile-compcert; both models: rstan-to-pystan. Final text reports builds or sampling still running; required artifacts absent.	Completion gate may not wait for required background work. Track process IDs and exit status; require artifact checks before finalization. Test cancellation and bounded waiting on fresh long-build tasks.
Astra: crack-7z-hash and mcmc-sampling-stan. Final text explicitly says password recovery or RStan installation is unfinished.	Separate command launch from completion. Measure unfinished-final rate, artifact validity and elapsed budget. Do not merely extend one arm's time limit.
Both models: protein-assembly asks to resolve FASTA placeholders and creates no deliverable.	Study noninteractive ambiguity handling. Distinguish defensible assumptions from contradictions needing clarification. Do not suppress legitimate safety or authorization boundaries.
Astra: pytorch-model-recovery saves a model, but verifier reports a forward-call interface error.	Validate the user-facing invocation contract, not only local loss/weight checks. Add fresh API-signature and serialization regression tasks.
Astra: sanitize-git-repo fails expected replacement check; uses 1.38M vs 0.26M tokens.	Inspect replacement scope and repeated verification. Instrument redundant searches and output-size growth before choosing an optimization.
Sol: mteb-leaderboard; Astra: mteb-retrieve produce the wrong expected answer.	Audit date, source, ranking and exact task contract. Do not hardcode verifier answers or assume causal failure from one trace.

Astra compile-compcert, build-pov-ray, fix-ocaml-gc and write-compressor have Python-download failures in verification. They must not be counted as confirmed harness-specific correctness defects. The failure taxonomy and individual hashes are in task-results.csv.

Efficiency needs outcome context



Below 1 = fewer tokens. Complete telemetry only; subsets are descriptive.

The Astra both-solved set contains 53 tasks: native 16,125,794 tokens versus Codex 13,265,337 (+21.6%). Sol's 56 both-solved tasks use 12,969,990 versus 16,394,885 (-20.9%). Neither selected subset establishes overall noninferiority or a causal efficiency advantage.

Largest native token excesses on tasks both solve

Model / task	80D tokens	Codex tokens	Excess
Astra / extract-moves-from-video	4,370,152	2,355,644	+2,014,508
Astra / mteb-leaderboard	3,257,438	2,049,362	+1,208,076
Astra / path-tracing-reverse	664,197	124,795	+539,402
Sol / mailman	1,009,471	530,408	+479,063
Sol / path-tracing-reverse	573,846	189,709	+384,137
Sol / schemelike-metacircular-eval	1,450,805	1,147,709	+303,096

Investigate tool-call repetition, context growth, rejected edits and repeated full-file reads in these traces. These are hypotheses to measure, not established causes. Token budgets should be assessed jointly with correctness; shorter unfinished runs are not successful efficiency improvements.

Telemetry and runner priorities

Priority	Change to test	Acceptance criterion
P0: verifier health	Prebuild/pin verifier dependencies; no live Python/package download during scoring. Keep task logic unchanged and record image digests.	Synthetic canary verifies runner availability before paid trials; setup failures cannot be silently recorded as solution failures. New protocol for changed environment.
P0: timeout usage	Flush native per-turn usage before timeout termination; retain provider usage provenance and partial/full status.	Two make-doom-for-mips native trials currently lack totals. Future forced-timeout tests preserve observed usage without calling partial usage complete.
P0: completion lifecycle	Track pending required processes and declared artifacts at finalization.	No final success while a required job is pending; correctly handles failed, canceled and long-running processes within the same budget.
P1: resource handling	Preflight disk and Docker health; cache eviction only for completed tasks. Preserve runtime separately from cleanup targets.	Resume skips scored cells; failure evidence and cap remain intact. Runtime removal never triggers repeated paid launch.
P1: comparable identity	Capture actual model, effort and service tier in each provider event, with fail-closed mismatches.	No inferred identity from requested flags alone; no fallback or unrecorded mixed model.
P1: efficiency instrumentation	Record per-turn token growth, tool name, duration and output size, with redaction.	Explain token regressions on held-out successes without reducing correctness or leaking credentials.

Provider attribution: seven failures have explicit capacity messages. One reservation failed locally before Harbor could launch. Verifier-network and package-manager failures are a separate category. Do not relabel all errors as provider failures.

370 ledger reservations contain 356 attempts with complete reported usage and 14 without complete usage (including the known zero-call local failure). Observed subtotals are lower bounds, not full-study cost comparisons. Synthetic preflights are outside this ledger.

A testable improvement plan

Step	Design and decision rule
1. Freeze evidence	Retain raw outcomes, failure logs and hashes. Use the public task table to reproduce every aggregate. Keep audit data out of agent prompts.
2. Isolate changes	Use a new commit and one intervention per ablation: lifecycle gate, contract validation, telemetry flush, then context efficiency. Preserve current successes as regressions.
3. New validation set	Use independent tasks with similar long-build, output-contract and ambiguity patterns. Randomize/interleave arms; fix models, efforts, tools, limits and environment before outcomes.
4. Declare evidence requirements	Choose sample size with prospective power analysis and an explicit noninferiority margin. Publish correctness first, then tokens on the identical eligible set. Do not relax the margin after seeing results.
5. Separate reliability from scoring	Keep a capped error ledger. Reverify a saved immutable solution only when the original environment/output is recoverable; do not substitute a new agent attempt for the old outcome.
6. Promote claims only on evidence	Require the full statistical gates and missingness sensitivity. Use actual billing for a money claim. Publish unfavorable results as well as wins.

Why no more paid runs were added

Existing data are sufficient to decide that the proposed claims are not established. Many containers were deleted after trials, so a fresh agent run would not be verification of the original solution. Historical cells include exhausted attempt caps. Selective reruns after seeing outcomes would bias the headline. A new predeclared validation study is the appropriate next step.

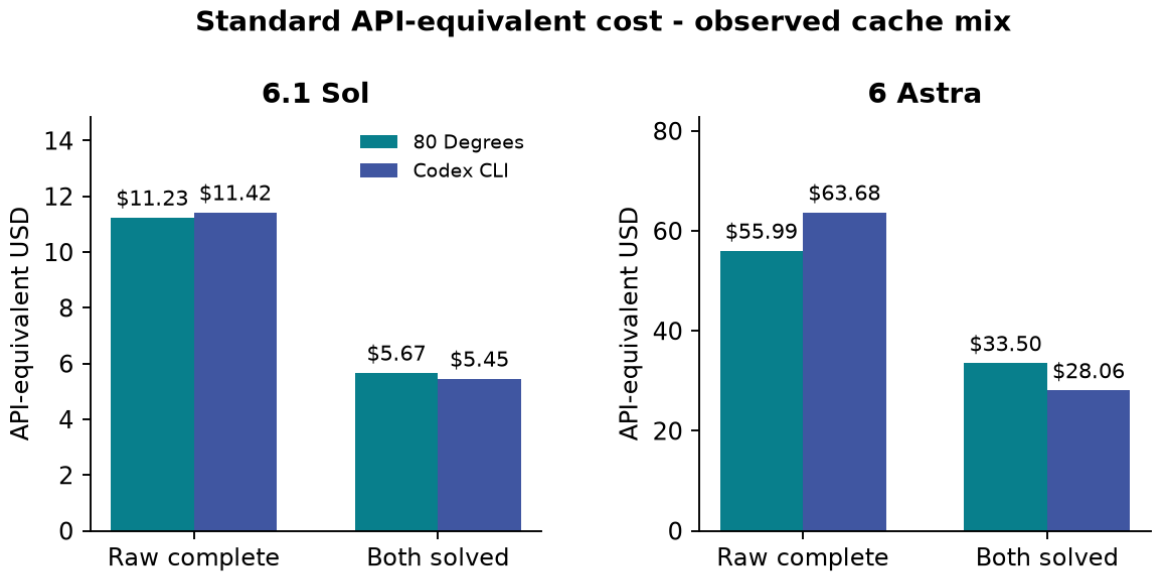
Trace provenance

Case studies are based on native final message events, paired raw results and verifier logs. Public rows include result/verifier SHA-256 hashes; private paired-evidence.json maps them to retained source files. Facts, hypotheses and proposed experiments are deliberately separate. No new competitor claims or SWE-bench results are implied.

Measured source base: 7a6e9e9acbd00eae1c0548a4c4f054b94c203faa
Measured adapter: 770667538eb64aae2169b4006aa41ee781ae994c
Engine: 028c1d7c7eb753421012bf96472a4ea040667f19
Codex CLI 0.163.0; Harbor 0.24.0. Full provenance and statistical methods accompany the investor report.

API-equivalent cost scorecard

Official prices checked 28 October 2026. Standard text-token rates applied equally to both harnesses; no regional or priority premium. This answers a modeled-cost question, not actual subscription spending.



Estimated text-token cost, not subscription charges. Subsets have different coverage.

Measured subset	Sol: 80D / Codex	Astra: 80D / Codex
Raw complete pairs (86 / 88)	$11.23 / $11.42	$55.99 / $63.68
Observed cost reduction	1.6% lower	12.1% lower
Resolved complete pairs (73 / 74)	$8.34 / $8.82	$45.93 / $44.91
Both-solved pairs (56 / 53)	$5.67 / $5.45	$33.50 / $28.06
Both-solved cost difference	80D 3.9% higher	80D 19.4% higher

Formula: (uncached input x input rate + cache reads x cached rate + output x output rate) / 1,000,000. Zero cache writes were recorded. Rates per million: Sol $2 / $0.10 / $10; Astra $10 / $1 / $50, respectively. Reasoning and cached input are counted once.

No observed request exceeded the 272K-input long-context threshold. Two native aggregate/event residuals are below that threshold and priced from aggregate totals. Missing usage, retries, historical reservations, tool charges and subscription fees are excluded. Rate normalization does not prove what an actual API replay would cost.

Raw paired bootstrap 95% cost-ratio intervals: Sol [0.834, 1.159], Astra [0.689, 1.105]. Both include 1. A lower observed modeled total is a descriptive result, not proven equal-correctness cost superiority. Both-solved subsets are selected by outcomes.

Sources: developers.openai.com/api/docs/pricing; developers.openai.com/api/docs/models/gpt-6.1-sol; developers.openai.com/api/docs/models/gpt-6-astra. Reproduce with tooling/price_usage.py; numeric source-trace audit is in pricing-audit.json.